

Contents

- Maintainer Guide
 - 1. Overview
 - Open Chiplet Atlas and the Open Chiplet Harness
 - What `tt-oca-sim` provides
 - 2. Prerequisites
 - 3. Repository Architecture
 - 4. Initial Setup
 - 4.1 Clone the Repository
 - 4.2 Verify Submodule State
 - 4.3 Configure and Build
 - 5. Core Maintainer Workflows
 - 5.1 Reviewing and Merging Pull Requests
 - 5.2 Bumping the CSML Submodule
 - 5.3 Creating a Release Tag
 - 5.4 Updating RELEASE_NOTES.md
 - 6. Adding a New Peripheral Model
 - 7. Removing or Archiving a Peripheral
 - 8. Onboarding a New Contributor (End-to-End)
 - 8.1 Information Needed
 - 8.2 Grant Repository Access
 - 8.3 Branch Protection
 - 8.4 Handoff to the Contributor
 - 8.5 Onboarding Checklist
 - 9. Submodule Management
 - Common submodule operations
 - CSML is not initialized
 - Detached HEAD / dangling commits in CSML
 - 10. CI/CD Pipeline
 - 10.1 Workflow Structure
 - 10.2 Dependency Caching

- 10.3 Enabling the CI Check as a Required Status
- 10.4 Viewing Results and Artifacts
- 10.5 Running CI Checks Locally
- 11. Build System Maintenance
 - 11.1 Directory Structure
 - 11.2 When to Update Build Scripts
 - 11.3 Testing Build Changes
- 12. Documentation Maintenance
 - 12.1 Markdown-to-PDF Workflow
 - 12.2 When to Update Docs
- 13. Access Control and Permissions
 - GitHub Role Summary
 - Recommended Permission Structure
 - Protecting the `main` Branch
 - CODEOWNERS
 - Verifying Permissions
- 14. Troubleshooting
- 15. Quick Reference
 - Submodule Commands
 - Release Commands
 - Build Commands
 - Documentation Commands

Maintainer Guide

Audience: Engineers responsible for maintaining the `tt-oca-sim` repository — the Tenstorrent Open Chiplet Atlas (OCA) Virtual Platform — and coordinating integration across SMC and SEP peripheral models, the SEP VP platform, firmware test infrastructure, and documentation.

Last updated: 2026-06-18

1. Overview

Open Chiplet Atlas and the Open Chiplet Harness

Open Chiplet Atlas (OCA) is Tenstorrent's chiplet-based System-in-Package (SiP) architecture. The **Open Chiplet Atlas Harness (OCAH)** is the hardware specification governing every OCA chiplet — defining the common infrastructure layer (AXI fabric topology, inter-chiplet protocols OCCP and OCTS, security boundaries, management interfaces) that all chiplets must implement.

Every OCA chiplet integrates two mandatory management subsystems:

- **SMC (System Management Controller)** — the per-chiplet firmware-driven RISC-V management engine (clock, voltage, reset, inter-chiplet communication, security fabric, telemetry, interrupt aggregation). Built around a 1–4 core Rocket RV64GC cluster; defined in OCAH Ch. 6.
- **SEP (Secure Enclave Processor)** — the per-chiplet OpenTitan-derived security enclave (AES, HMAC, KMAC, OTBN, Key Manager, Lifecycle Controller, secure boot). Communicates with the SMC via `sep_axi_in` and a dedicated mailbox interface.

What `tt-oca-sim` provides

`tt-oca-sim` provides **SystemC/TLM-2.0 virtual platform simulation** for both OCAH subsystems:

Subsystem	What is provided
SEP	Full, runnable Virtual Platform (<code>sep-vp</code>) — models all SEP peripherals, executes RISC-V VeeR EL2 firmware, used for pre-silicon DV and firmware development
SMC	TLM-2.0 IP model library — individual models for each SMC peripheral (PLIC, CLINT, CPU cluster, reset unit, bootrom, scratchpad, I3C), with unit tests and documentation

The repository contains:

Path	Contents
<code>sep/peripherals/</code>	All SEP IP peripheral models (AES, HMAC, KMAC, OTBN, ...)
<code>sep/cpu/</code>	VeeR EL2 ISS + TLM-2.0 wrapper
<code>sep/utils/csml/</code>	Core SystemC Model Library (submodule — Vayavya CSML)
<code>sep/utils/paged-memory/</code>	PagedMemory header-only sparse-storage engine
<code>smc/peripherals/</code>	SMC peripheral models (PLIC, CLINT, bootrom, reset unit, I3C, ...)
<code>smc/cpu_cluster/</code>	SMC CPU cluster model (1–4 RV64GC, Whisper ISS backend)
<code>vp/</code>	SEP VP platform (wires all SEP models into <code>sep-vp</code> executable)
<code>sw/</code>	Firmware and DV test suites
<code>doc/</code>	Architecture, design, and test-plan documents
<code>scripts/</code>	Markdown→PDF conversion toolchain
<code>cmake/</code>	<code>FindSystemC.cmake</code> , <code>FindCCI.cmake</code>

Your role as maintainer:

- Review and merge Pull Requests for both SMC and SEP subsystems
- Bump the `sep/utils/csml` submodule when the upstream CSML library is updated
- Create release tags and update `RELEASE_NOTES.md`
- Add or remove peripheral models as the project evolves
- Keep build scripts, CMakeLists, and CI in sync
- Onboard new contributors and manage GitHub permissions
- Ensure all peripheral unit tests and firmware VP tests remain green

2. Prerequisites

- SSH access to `github.com` (configured key in `~/.ssh/`)
- Maintainer or Admin role on `github.com/tenstorrent/tt-oca-sim`
- Git ≥ 2.25
- CMake ≥ 3.24
- C++ compiler: GCC 9+ (C++17) or GCC 11+ (C++20)

- SystemC 3.0.2 and CCI 1.0.1 installed locally
 - pandoc (for documentation PDF generation): `brew install pandoc`
 - Google Chrome (for Markdown→PDF rendering via `scripts/md-to-pdf.sh`)
-

3. Repository Architecture

```
tt-oca-sim/
├── .gitmodules                ← CSML submodule declaration
├── Makefile                  ← top-level convenience build
├── README.md
├── RELEASE_NOTES.md
├── cmake/
│   ├── FindSystemC.cmake
│   └── FindCCI.cmake
├── sep/                      ← SEP IP models
│   ├── peripherals/
│   │   ├── aes/
│   │   ├── aon_timer/
│   │   ├── csrng/
│   │   ├── edn/
│   │   ├── efuse/
│   │   ├── entropy_src/
│   │   ├── gpio/
│   │   ├── hmac/
│   │   ├── key_manager/
│   │   ├── kmac/
│   │   ├── lifecycle_ctrl/
│   │   ├── mailbox/
│   │   ├── otbn/
│   │   ├── secure_dma/
│   │   ├── sep_memory/
│   │   ├── spi_controller/
│   │   ├── spi_flash/
│   │   ├── uart_16550/
│   │   └── run_all_peripherals.sh
│   ├── cpu/
│   │   ├── VeeR-ISS/
│   │   ├── VeeR-ISSTlm/
│   │   └── run_tests.sh
│   └── utils/
│       ├── csml/              ← Git submodule (Vayavya CSML)
│       └── paged-memory/
├── smc/                      ← SMC/NW peripheral models
│   ├── run_all_smc_tests.sh   ← orchestrator: Release+ASAN+Coverage+CTest for
│   └── all SMC IPs
│       ├── peripherals/
│       │   ├── bootrom/
│       │   ├── clint/
│       │   ├── i3c_controller/
│       │   ├── plic/
│       │   └── reset_unit/
```

```

├── scratchpad_ram/
├── smc_fabric/           ← SMC AXI fabric model + testbench
├── cpu_cluster/
├── cmake/
│   └── SmcSystemCStd.cmake ← auto-detect SystemC C++ standard
├── vp/
│   ├── CMakeLists.txt
│   ├── configure_vp.sh   ← configure CMake + export build env
│   └── platform/
│       ├── infra/        ← bus, PLIC, CLINT, ELF loader
│       └── sep/
│           ├── main.cpp
│           ├── och_sep_ss.hpp ← top-level platform module
│           ├── inc/
│           └── config/
│               ├── accelera_config.ini
│               └── veeriss_config.json
├── sw/
│   ├── sep-vp-tests/     ← Vayavya peripheral verification tests
│   └── tt-oca-hw-main/   ← TT DV + firmware tests
│       ├── bin/sep_fw_standalone.sh
│       ├── dv/sep/tests/
│       └── fw/sep/tests/
├── scripts/
│   ├── md-to-pdf.sh
│   └── md-pdf.css
├── doc/
│   ├── maintainer-guide.md ← this file
│   ├── component-developer-guide.md
│   ├── 01_SMC_Architecture.md
│   ├── 02_SMC_IP_LowLevel_Design.md
│   ├── 03_SMC_Test_Plan.md
│   └── ...

```

The `sep/utils/csml` directory is a Git submodule pointing to the Vayavya CSML library at [git@github.com:Vayavya-Labs/CSML.git](https://github.com/Vayavya-Labs/CSML.git). All other directories are part of the main repository.

4. Initial Setup

4.1 Clone the Repository

```
git clone --recurse-submodules \
  git@github.com:tenstorrent/tt-oca-sim.git
cd tt-oca-sim
```

4.2 Verify Submodule State

```
# Shows each submodule's current commit SHA and status
git submodule status

# Prefix meanings:
# '-' submodule not yet initialized
# '+' submodule is at a different commit than pinned
# ' ' submodule matches the pin ← correct
```

If you cloned without `--recurse-submodules`:

```
git submodule update --init --recursive
```

The top-level `Makefile` runs `git submodule update --init --recursive --quiet` automatically before every build, so the VP always builds against the pinned CSML commit.

4.3 Configure and Build

Edit `vp/configure_vp.sh` to set `SYSTEMC_HOME`, `CCI_HOME`, `BOOST_ROOT`, and `OPENSSL_ROOT` to your local install paths. Then:

```
# C++17 (default)
cd vp && ./configure_vp.sh && cd build && make sep-vp

# C++20
CMAKE_CXX_STANDARD=20 cd vp && ./configure_vp.sh && cd build && make sep-vp

# Or from repo root (uses top-level Makefile)
SYSTEMC_HOME=/path/to/systemc make sep-vp
```

Output binary: `vp/build/bin/sep-vp`

C++ standard note: The `smc/cmake/SmcSystemCStd.cmake` helper probes the linked SystemC library for its compile-time C++ standard and automatically sets `CMAKE_CXX_STANDARD` to match. Always point `SYSTEMC_HOME` at a SystemC build compiled with the standard you intend to use — mixing standards fails at link time with an `sc_api_version_*` undefined-symbol error.

5. Core Maintainer Workflows

5.1 Reviewing and Merging Pull Requests

All changes to `main` go through Pull Requests. As maintainer:

1. Confirm the PR targets `main` (or the active release branch).
2. Check that all CI checks pass (build + unit tests + lint).
3. Verify that any new peripheral follows the standard layout (see [Section 6](#)).
4. Ensure test coverage is adequate — new models must ship with unit tests under `<subsystem>/peripherals/<ip>/test/`.
5. Merge using **"Squash and merge"** for single-commit changes or **"Merge commit"** for multi-commit feature branches — keep history readable.

```
# After the PR merges, sync your local main
git checkout main
git pull origin main
git submodule update --init --recursive
```

5.2 Bumping the CSML Submodule

When `Vayavya-Labs/CSML` releases a new version or a fix is needed:

```

cd sep/utils/csml
git fetch origin
git checkout <new-tag-or-commit>
cd ../../..

# Review what changed
git diff --submodule

# Commit the new pin
git add sep/utils/csml
git commit -m "chore(csml): bump CSML to <new-tag>

Includes: <brief description of relevant changes>"
git push origin main

```

Policy: Only bump to tagged releases of CSML unless an untagged fix is urgently required. Always document the reason in the commit message.

5.3 Creating a Release Tag

Tag the repository to create a reproducible snapshot that covers the exact versions of all source files and the pinned CSML commit.

```

cd tt-oca-sim
git checkout main
git pull origin main
git submodule status      # verify all submodules are at correct pins

# Create an annotated tag
git tag -a v2.2.0 -m "Release v2.2.0

Highlights:
- <summary of major changes>
- CSML: <csml-tag>
- Peripheral additions: <list>
- Tests: <status>"

git push origin v2.2.0

```

Tag naming convention: `v<MAJOR>.<MINOR>.<PATCH>`

To recreate the exact system state from a tag:

```
git clone --recurse-submodules --branch v2.2.0 \
git@github.com:tenstorrent/tt-oca-sim.git
```

5.4 Updating RELEASE_NOTES.md

Update `RELEASE_NOTES.md` **before** creating the release tag:

- Increment the version number and release date.
- List all included peripheral models.
- Update the testing status section.
- Record any new limitations or resolved issues.
- Update the GCC/C++ compatibility table if compiler coverage changed.

Commit the updated release notes as part of the release commit:

```
git add RELEASE_NOTES.md
git commit -m "chore: prepare release v2.2.0"
git push origin main
# Then tag (see 5.3)
```

6. Adding a New Peripheral Model

Peripheral models live under either `sep/peripherals/<ip>/` (SEP — OpenTitan-derived security IP) or `smc/peripherals/<ip>/` (SMC — management/infrastructure IP per OCAH Ch. 6). New models must follow the standard layout:

```
<ip>/
├─ CMakeLists.txt
├─ README.md
├─ run_tests.sh          ← wraps cmake + ctest with --asan / --coverage flags
├─ include/              ← public headers
├─ src/                  ← implementation
├─ test/                 ← CTest-registered unit tests
├─ doc/
│   └─ 01_<IP>_Specification/
│   └─ 02_<IP>_HighLevel_Design.md
│   └─ 03_<IP>_Test_Plan.md
```

Steps:

1. Create the directory and populate it following the layout above.
2. Register the model in the parent `run_all_peripherals.sh`:

```
# sep/peripherals/run_all_peripherals.sh
run_peripheral "new-ip"
```

3. Wire the model into the VP platform (`vp/platform/sep/och_sep_ss.hpp` and `vp/platform/sep/main.cpp`) if it should be instantiated in the full-system build.
4. Add CCI parameters for the new model to `vp/platform/sep/config/accellera_config.ini`.
5. Update `RELEASE_NOTES.md` — add the new IP to the "Included Models" list.
6. Update `doc/02_SMC_IP_LowLevel_Design.md` or the equivalent architecture doc to document the new model's register map and TLM interface.
7. Open a PR and ensure all CI checks pass before merging.

7. Removing or Archiving a Peripheral

```
# Remove the directory
git rm -r <subsystem>/peripherals/<ip>/

# Remove from run_all_peripherals.sh
# Edit sep/peripherals/run_all_peripherals.sh or smc equivalent

# Remove from VP platform wiring (och_sep_ss.hpp / CMakeLists.txt)

# Update RELEASE_NOTES.md — move the IP to a "Removed Models" section

git commit -m "chore(<ip>): remove <ip> peripheral model

Reason: <brief justification>"
git push origin main
```

If the model may be needed again, consider keeping the directory but removing VP wiring and adding a `DEPRECATED.md` at the model root.

8. Onboarding a New Contributor (End-to-End)

8.1 Information Needed

Field	Example
GitHub username	@jane-doe
Area of work	SEP peripheral models / VP platform / firmware tests
Access level needed	Write (contributor) or Maintain

8.2 Grant Repository Access

GitHub → github.com/tenstorrent/tt-oca-sim → **Settings** → **Collaborators and teams** → **Add people**

Role	Capability
Write	Push branches, open/merge PRs (if branch protection allows)
Maintain	Manage issues, PRs, and some settings — cannot change branch protection
Admin	Full repository control

Grant **Write** access for standard contributors. Reserve **Maintain/Admin** for integration leads.

8.3 Branch Protection

Verify that the `main` branch is protected:

GitHub → **Settings** → **Branches** → **main**

Rule	Setting
Require pull request reviews	✓ (1 required reviewer)
Require status checks to pass	✓ (all CI jobs)
Restrict who can push	Maintainers only
Allow force pushes	✗

8.4 Handoff to the Contributor

Send the new contributor the following:

```
Repository: git@github.com:tenstorrent/tt-oca-sim.git
Access: Write

Clone:
  git clone --recurse-submodules git@github.com:tenstorrent/tt-oca-sim.git
  cd tt-oca-sim

Build:
  cd vp && ./configure_vp.sh && cd build && make sep-vp

Run all peripheral tests from repo root:
  sep/peripherals/run_all_peripherals.sh

Refer to:
  - README.md           - full build and run instructions
  - doc/component-developer-guide.md - day-to-day development workflow
  - RELEASE_NOTES.md    - current feature/test status
```

8.5 Onboarding Checklist

- ☐ GitHub access granted (Write or Maintain)
- ☐ Contributor confirmed they can clone and build successfully
- ☐ Contributor has read `doc/component-developer-guide.md`
- ☐ CSML submodule initializes correctly (`git submodule status` shows no `-` prefix)
- ☐ Contributor has run at least one peripheral test end-to-end

9. Submodule Management

The repository has one Git submodule:

Path	Remote	Purpose
<code>sep/utils/csml</code>	<code>git@github.com:Vayavya-Labs/CSML.git</code>	Core SystemC Model Library — register modelling, CCI params, logging

Common submodule operations

```
# Check current pin
git submodule status

# Initialize after plain clone
git submodule update --init --recursive

# Update to a new CSML tag
cd sep/utils/csml
git fetch origin
git checkout <tag>
cd ../../..
git add sep/utils/csml
git commit -m "chore(csml): bump CSML to <tag>"
git push origin main

# See what changed relative to the pinned commit
git diff --submodule

# Run a command inside all submodules
git submodule foreach 'git log --oneline -3'
```

CSML is not initialized

Symptom: `git submodule status` shows a leading `-` before the SHA:

```
-56d9d2dde9045e2400a46d4f0fbdea0f2f905d34 sep/utils/csml
```

The `-` means the submodule has never been initialized in this working tree — the `sep/utils/csml/` directory exists but is empty. This happens after a plain `git clone` without `--recurse-submodules`.

Fix:

```
git submodule update --init --recursive
```

Verify — re-run `git submodule status`. The `-` prefix should be gone and replaced with a space (correct) or `+` (initialized but at a different commit than pinned):

```
56d9d2dde9045e2400a46d4f0fbdea0f2f905d34 sep/utils/csml (heads/main)
```

If `update --init` fails (e.g. the `.git/modules/` entry is corrupted), force a clean re-initialization:

```
git submodule deinit sep/utils/csml
git submodule update --init sep/utils/csml
```

Detached HEAD / dangling commits in CSML

If someone committed inside the submodule while it was in detached HEAD state, the commits are not lost. Recover them:

```
cd sep/utils/csml
git reflog                                # find the dangling commit SHA
git branch rescue-branch <sha>           # name it so GC won't collect it
git checkout main
git cherry-pick <sha>                     # bring the work onto main
git push origin main
```

10. CI/CD Pipeline

The CI pipeline lives at `.github/workflows/ci.yml` and runs automatically on every Pull Request targeting `main` and on every push to `main`.

10.1 Workflow Structure

The workflow has five parallel jobs (all run on `ubuntu-22.04`):

Job	What it does
<code>build-deps</code>	Builds SystemC 3.0.2 and CCI 1.0.1 from source; results are cached so subsequent runs skip the build
<code>sep-vp</code>	Checks out the repo (with submodules), restores the deps cache, and builds <code>sep-vp</code> via <code>vp/configure_vp.sh</code> + <code>make sep-vp</code>
<code>sep-unit-tests</code>	Restores deps cache and runs <code>sep/peripherals/run_all_peripherals.sh</code> (Release + ASAN + Coverage + CTest for every SEP peripheral)
<code>smc-unit-tests</code>	Restores deps cache and runs <code>smc/run_all_smc_tests.sh</code> for all peripherals (Release + ASAN + Coverage + CTest; uploads <code>smc-peripheral-logs</code> artifact)
<code>smc-fabric-tests</code>	Restores deps cache (SystemC only) and runs <code>smc/run_all_smc_tests.sh smc_fabric</code> (Release + ASAN + Coverage + CTest; uploads <code>smc-fabric-logs</code> artifact)

`sep-vp`, `sep-unit-tests`, `smc-unit-tests`, and `smc-fabric-tests` all declare `needs: build-deps` so they start only after the dependency cache is warm, then run in parallel.

Note: The `smc/cpu_cluster` tests are **not** run in CI because they require the Tenstorrent-internal Whisper ISS (`WHISPER_HOME`). Run them locally once `WHISPER_HOME` is set.

10.2 Dependency Caching

SystemC and CCI are each cached under a key that encodes OS + compiler version + C++ standard:

```
systemc-3.0.2-Linux-cxx20
cci-1.0.1-Linux-cxx20
```

A cache hit means the `build-deps` job completes in seconds. A miss (new OS image, version bump, or standard change) triggers a full rebuild and writes a new cache entry.

To **force a cache rebuild** (e.g. after bumping `SYSTEMC_VERSION`):

1. Update `SYSTEMC_VERSION` or `CCI_VERSION` in `.github/workflows/ci.yml`.
2. Push — the changed version string makes the cache key miss.

10.3 Enabling the CI Check as a Required Status

Once the first workflow run completes successfully, make the CI a **required check** on the `main` branch so PRs cannot be merged unless all jobs pass:

GitHub → **Settings** → **Branches** → `main` (edit rule) → **Require status checks to pass before merging** → search for and add:

- `Build SystemC + CCI`
- `Build sep-vp`
- `SEP peripheral unit tests`
- `SMC peripheral unit tests`
- `SMC Fabric unit tests`

10.4 Viewing Results and Artifacts

- CI results appear on every PR under the **Checks** tab.
- Per-peripheral test logs are uploaded as CI artifacts (retained 7 days) — download from **Actions** → the workflow run → **Artifacts**:

- `sep-peripheral-logs` — SEP peripheral Release/ASAN/Coverage/CTest logs and `logs/Full_result.log` summary table
 - `smc-peripheral-logs` — SMC peripheral Release/ASAN/Coverage/CTest logs and `smc/logs/Full_result.log` summary table
 - `smc-fabric-logs` — SMC Fabric Release/ASAN/Coverage/CTest logs and `smc/logs/Full_result.log` summary table
- If a job fails, click the job name in the PR Checks tab to see the full log.

10.5 Running CI Checks Locally

You can reproduce any CI job locally before pushing. `configure_vp.sh` auto-discovers `SYSTEMC_HOME`, `CCI_HOME`, `BOOST_ROOT`, and `OPENSSL_ROOT` — no exports needed:

```
# sep-vp build
CMAKE_BUILD_TYPE=Release bash vp/configure_vp.sh
make sep-vp -C vp/build -j$(nproc)

# SEP unit tests (Release + ASAN + Coverage + CTest for all SEP peripherals)
sep/peripherals/run_all_peripherals.sh

# SMC peripheral unit tests (Release + ASAN + Coverage + CTest; all except
cpu_cluster)
smc/run_all_smc_tests.sh

# SMC Fabric tests only
smc/run_all_smc_tests.sh smc_fabric

# Run a single SMC peripheral
smc/run_all_smc_tests.sh plic
```

The `smc/run_all_smc_tests.sh` orchestrator produces the same structured output as `sep/peripherals/run_all_peripherals.sh`: per-IP `Full_result.log` files and a top-level `smc/logs/Full_result.log` summary table with columns `IP | Release | ASAN | Coverage | CTest | Coverage%`.

11. Build System Maintenance

11.1 Directory Structure

```
Makefile          ← top-level: sep-vp, submodule-init, clean
vp/
├─ configure_vp.sh ← exports SYSTEMC_HOME, CCI_HOME, BOOST_ROOT,
OPENSSL_ROOT
├─ CMakeLists.txt  ← VP target definition
├─ platform/
│   └─ CMakeLists.txt
│       └─ infra/      ← bus, PLIC, CLINT, ELF loader
│           └─ sep/     ← SEP platform (och_sep_ss, main, config)
cmake/
├─ FindSystemC.cmake ← locates SystemC install
├─ FindCCI.cmake     ← locates CCI install
smc/cmake/
└─ SmcSystemCStd.cmake ← auto-detects SystemC C++ standard from linked library
```

11.2 When to Update Build Scripts

Event	Action
New peripheral added	Add it to <code>vp/platform/sep/CMakeLists.txt</code> if VP-integrated; add <code>add_subdirectory</code> in the appropriate <code>CMakeLists.txt</code>
Peripheral changes directory layout	Update <code>CMakeLists.txt</code> source/include paths
New SystemC/CCI version adopted	Update <code>SYSTEMC_HOME</code> / <code>CCI_HOME</code> defaults in <code>vp/configure_vp.sh</code>
C++ standard changed	Update <code>CMAKE_CXX_STANDARD</code> default in <code>vp/configure_vp.sh</code> and verify <code>SmcSystemCStd.cmake</code> detects it correctly
New platform target	Add a new target in <code>vp/CMakeLists.txt</code>

11.3 Testing Build Changes

Always test both C++17 and C++20 before pushing build script changes:

```
# Default build (C++20, auto-discovered paths)
bash vp/configure_vp.sh
make sep-vp -C vp/build

# Explicit C++17 fallback
CMAKE_CXX_STANDARD=17 bash vp/configure_vp.sh
make sep-vp -C vp/build

# Run all peripheral unit tests
sep/peripherals/run_all_peripherals.sh
```

12. Documentation Maintenance

12.1 Markdown-to-PDF Workflow

All documentation is authored in Markdown and rendered to PDF using the pipeline:

Markdown → pandoc (HTML5) → Chrome headless → PDF

The toolchain lives in `scripts/`:

```
scripts/
├── md-to-pdf.sh      ← main conversion script
└── md-pdf.css       ← stylesheet (GitHub-flavoured styling)
```

Usage:

```
# Convert a single file (output in same directory as input)
scripts/md-to-pdf.sh doc/maintainer-guide.md

# Convert with a custom output directory
scripts/md-to-pdf.sh -o /tmp/pdfs doc/maintainer-guide.md

# Convert all Markdown files under doc/
scripts/md-to-pdf.sh doc/

# Or use the per-file export script for component-developer-guide
bash doc/export-pdf.sh
```

Requirements:

```
brew install pandoc
# Google Chrome must be installed at /Applications/Google Chrome.app
# or set CHROME_BIN=/path/to/chrome
```

12.2 When to Update Docs

Event	Documents to update
New peripheral added	<code>doc/02_SMC_IP_LowLevel_Design.md</code> , <code>RELEASE_NOTES.md</code> , <code>README.md</code> component table
New firmware test added	<code>README.md</code> test table
Build instructions change	<code>README.md</code> , <code>doc/component-developer-guide.md</code>
New release	<code>RELEASE_NOTES.md</code> , regenerate all PDFs
Maintainer/contributor changes	<code>doc/maintainer-guide.md</code> permission tables

After updating any Markdown document, regenerate its PDF:

```
scripts/md-to-pdf.sh doc/<updated-file>.md
git add doc/<updated-file>.md doc/<updated-file>.pdf
git commit -m "docs: update <document> for <reason>"
```

13. Access Control and Permissions

GitHub Role Summary

Role	Capabilities
Admin	Full control — manage members, settings, branch protection, delete repo
Maintain	Manage issues/PRs, push to non-protected branches, manage some settings
Write	Push branches, open PRs; cannot merge without PR approval
Read	Read-only access to code, issues, and CI results

Recommended Permission Structure

Level	Who	Role	Purpose
Repository	Integration lead (@pdroy)	Admin	Full control, release tags, branch protection
Repository	Core contributors	Write	Feature development, can merge own PRs after review
Repository	External reviewers	Read	Code review, visibility

Protecting the `main` Branch

GitHub → **Settings** → **Branches** → **Branch protection rules** → Edit `main`

Rule	Recommended setting
Require a pull request before merging	✓
Required number of approvals	1
Dismiss stale pull request approvals	✓
Require status checks to pass	✓
Require branches to be up to date	✓
Restrict who can push to matching branches	Admins / Maintainers only
Allow force pushes	✗
Allow deletions	✗

CODEOWNERS

Add a `CODEOWNERS` file at the repository root to require approval from the appropriate domain expert when specific paths change:

```
# .github/CODEOWNERS
*                @pdroy
sep/peripherals/ @pdroy
smc/peripherals/ @pdroy
vp/              @pdroy
doc/             @pdroy
scripts/         @pdroy
sep/utils/csml   @pdroy
```

Verifying Permissions

```
# A contributor should be able to push a feature branch
git push -u origin feature/test-access

# They should NOT be able to push directly to main
git push origin main      # fails - "protected branch"
```

14. Troubleshooting

Submodule shows **(modified content)** but no files changed

Build artifacts inside the submodule directory cause this. Clean and reset:

```
cd sep/utils/csml
git clean -fd
git checkout .
cd ../../..
```

git submodule update fails with "not a git repository"

The **.git/modules/** entry is corrupted. Re-initialize:

```
git submodule deinit sep/utils/csml
git submodule update --init sep/utils/csml
```

Build fails: **Undefined symbols: sc_api_version*_cxx202002L**

The VP was compiled against a SystemC built with C++17 while **CMAKE_CXX_STANDARD=20** (or vice versa). Fix by pointing **SYSTEMC_HOME** at the SystemC build that matches the C++ standard you are using:

```
SYSTEMC_HOME=/path/to/systemc-c++20 CMAKE_CXX_STANDARD=20 bash vp/configure_vp.sh
```

CCI configuration issues at runtime

Check **vp/platform/sep/config/accellera_config.ini** — all CCI parameters must match the module hierarchy. A typo in a parameter path is silently ignored by CCI but leaves the model at its default value.

Firmware test: VP binary not found

```
make sep-vp # build from repo root
```

Firmware test: toolchain not found

```
export RISCV_TOOLCHAIN_PATH=/path/to/riscv-gnu-toolchain
```

run_all_peripherals.sh hangs on a particular peripheral

Interrupt the hung test (**Ctrl-C**), enter the peripheral directory, and run its tests manually with verbose output:

```
cd sep/peripherals/<ip>
./run_tests.sh --debug
# or:
cd build && ctest -V --rerun-failed
```

Chrome not found when generating PDFs

```
export CHROME_BIN="/Applications/Google Chrome.app/Contents/MacOS/Google Chrome"
scripts/md-to-pdf.sh doc/maintainer-guide.md
```

15. Quick Reference

Submodule Commands

Task	Command
Clone everything	<code>git clone --recurse-submodules git@github.com:tenstorrent/tt-oca-sim.git</code>
Initialize after plain clone	<code>git submodule update --init --recursive</code>
Check submodule status	<code>git submodule status</code>
Bump CSML to a new tag	<code>cd sep/utils/csml && git fetch && git checkout <tag> && cd ../../.. && git add sep/utils/csml</code>
Show submodule diff	<code>git diff --submodule</code>
Run command in all submodules	<code>git submodule foreach 'git log --oneline -3'</code>

Release Commands

Task	Command
Create release tag	<code>git tag -a v2.2.0 -m "Release v2.2.0"</code>
Push tag	<code>git push origin v2.2.0</code>
List tags	<code>git tag -l -n1</code>
Clone at a tag	<code>git clone --recurse-submodules --branch v2.2.0 git@github.com:tenstorrent/tt-oca-sim.git</code>

Build Commands

Task	Command
Configure VP (default C++20)	<code>cd vp && ./configure_vp.sh</code>
Configure VP (C++17 fallback)	<code>CMAKE_CXX_STANDARD=17 bash vp/configure_vp.sh</code>
Build sep-vp	<code>make sep-vp -C vp/build</code>
Build from repo root	<code>make sep-vp</code>
Clean build	<code>make clean && make sep-vp</code>
Run all peripheral tests	<code>sep/peripherals/run_all_peripherals.sh</code>
Run one peripheral's tests	<code>cd sep/peripherals/<ip> && ./run_tests.sh</code>
Run with AddressSanitizer	<code>cd sep/peripherals/<ip> && ./run_tests.sh --asan</code>
Run the VP	<code>vp/build/bin/sep-vp</code> <code>vp/platform/sep/config/accellera_config.ini <fw.elf></code>

Documentation Commands

Task	Command
Convert one Markdown file to PDF	<code>scripts/md-to-pdf.sh doc/<file>.md</code>
Convert all docs to PDF	<code>scripts/md-to-pdf.sh doc/</code>
Export component-developer-guide	<code>bash doc/export-pdf.sh</code>

For component development workflows (branching, committing, PRs, testing), see the [Component Developer Guide](#).